# BA EndTerm

**Name:** Shinjini Dasgupta

**Section:** 3

**Roll No:** FT273034

**Problem Statement**: The Indian Cricket Team's management wants to understand if the pre-match decisions have any impact on the final result. Given the independent variables of match are applied, we have to predict whether India will win or lose the match.

**Targets**:

- 1 – India lost the match
- 0 – India won the match

**Who uses this data**: The Indian Cricket Team's management uses this data to decide what kind of pre-match decisions can be taken and which decisions should be changed by predicting the outcome (India will win or lose) and invest on the resources accordingly.

**Type 1 Error**: It means False Positive. The Model predicts that India will win the match but actually India will lose the match. So, it will lead to waste of resources. The management will think that all factors and decisions will eventually lead to a win but it will not.

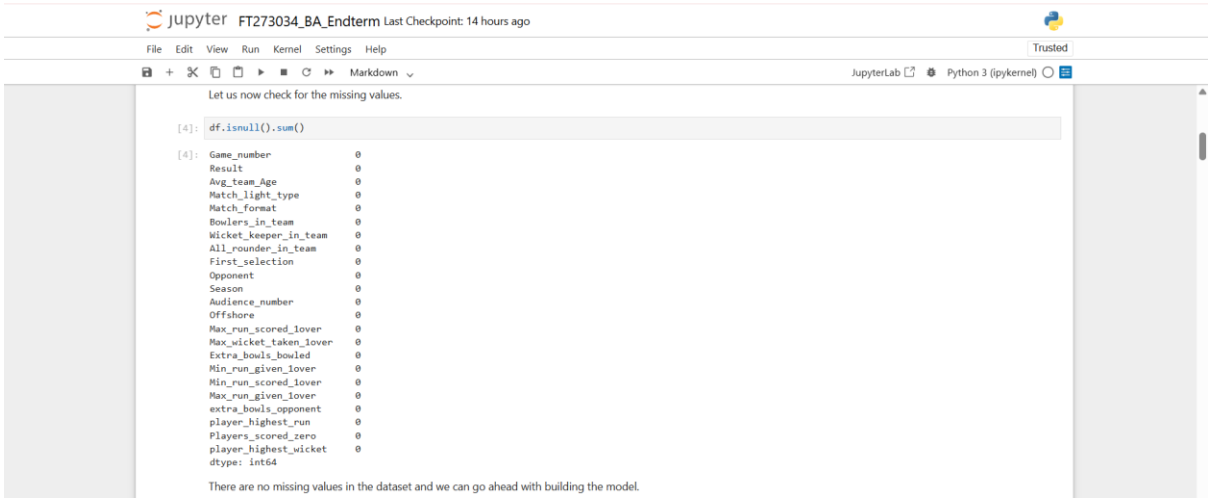
**Type 2 Error**: It means False Negative. The Model predicts that India will lose the match but actually India will win. This means that even after taking the wrong decisions, India will win the match so the management will not know which decisions led to the outcome. This is less costly as there will be less loss of resources.

In this case, Type 1 error seems to be more costly that Type 2 error as even after spending time and resources for impacting the result, India loses the match. The wrong prediction of the model showing that India will win provides false hopes to the management.

## Data Preparation

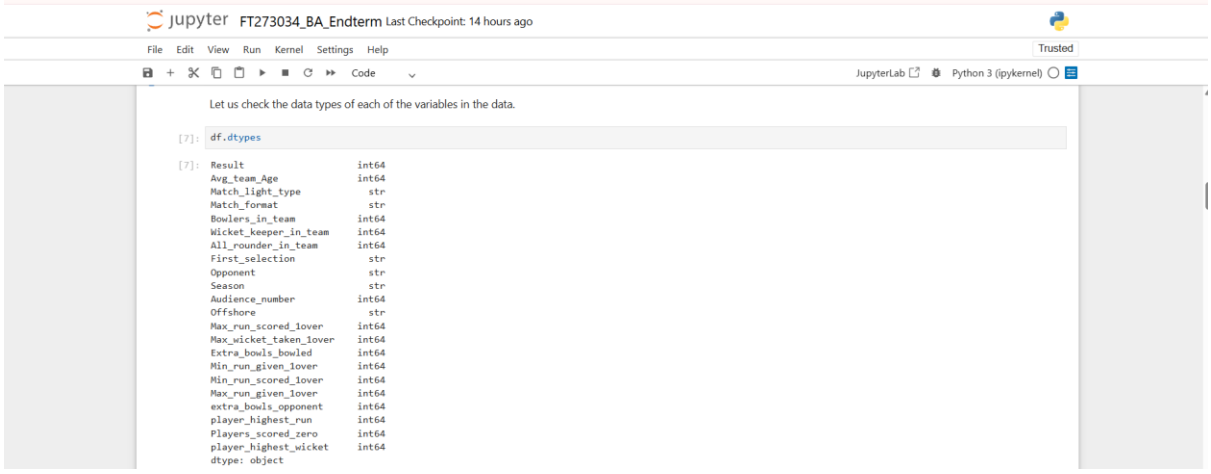
### Missing Data Check

Initial:




There are no missing values in the dataset and we can go ahead with building the model.

Here the Game_number variable is dropped as it is a unique number for each game and does not significantly affect the model's ability to perform prediction.




After dropping Game_number, we can see that many variables have object data which we will be encoding using One-Hot encoding where the categorical values are split into multiple variables having Boolean variables only. This is performed since the model can only process numerical or Boolean data.

We are prepping the data by segregating them into Target and independent variables to run this model going forward

```
[8]: #We are prepping the data by segregating them into Target and independent variables to runt his model going forward

     # Let us define the X(predictor) and Y(target) variables

     X = df.drop("Result" , axis=1)

     Y = df.pop("Result")
     #we have made a copy of the data frame as the 'pop' function removes that particular variable from the data frame and stores
     # in another variable

[9]: # This line of code is to perform one hot encoding for Categorical Features
     X = pd.get_dummies( X, drop_first = False )

[10]: list(X.columns)
```

We are then defining the X(predictor) and Y(target) variables.

**Splitting and Training the Data**:

We are splitting the data via 70:30 split where 70% will be used to train the model and 30% will be used for testing the prediction of the model which was trained.

### 1.1. Split the data into Train and Test.

Before building the model we should split the data into Train and Test. We will thus build a model on the training data and use this model to predict on the test data.

```
Before splitting the data, we shall make a copy of the data frame.
```

```
[13]: data = df.copy()
      data.head()
      #Just keeping the copy of the original dataset.
```

[13]:

| | Avg_team_Age | Match_light_type | Match_format | Bowlers_in_team | Wicket_keeper_in_team | All_rounder_in_team | First_selection | Opponent | Season | Audience_numbe |
|---|---|---|---|---|---|---|---|---|---|---|
| **0** | 18 | Day | ODI | 3 | 1 | 3 | Bowling | Srilanka | Summer | 994 |
| **1** | 24 | Day | T20 | 3 | 1 | 4 | Batting | Zimbabwe | Summer | 840 |
| **2** | 24 | Day and Night | T20 | 3 | 1 | 2 | Bowling | Zimbabwe | Summer | 1314 |
| **3** | 24 | Day | ODI | 2 | 1 | 2 | Bowling | Kenya | Summer | 735 |
| **4** | 24 | Night | ODI | 1 | 1 | 3 | Bowling | Srilanka | Summer | 1332 |

5 rows × 21 columns

```
[14]: from sklearn.model_selection import train_test_split

      X_train, X_test, Y_train, Y_test = train_test_split(X, Y, test_size=.30, random_state=1)
```

Now that we have split the data into Train and Test, let us go ahead and build our Decision Tree Model.

**Initial Decision Tree without any Pruning:**

- We set the initial conditions for the tree to be min_samples_split=2, min_samples_leaf=1. This was done so that we can have as many branches and splits in the tree as possible to understand the model behaviour.

- Feature Importance:

```
[18]: pd.Series(dt_model.feature_importances_,index=X_train.columns).sort_values(ascending=False)
```

```
[18]: Audience_number                0.179453
      player_highest_run             0.147419
      Extra_bowls_bowled             0.064608
      Players_scored_zero            0.057469
      Offshore_No                    0.054086
      extra_bowls_opponent           0.052714
      All_rounder_in_team            0.048938
      Season_Summer                  0.046588
      Min_run_scored_1over           0.044403
      Max_run_scored_1over           0.043655
      Opponent_South Africa          0.037016
      Bowlers_in_team                0.036900
      Max_run_given_1over            0.026936
      Max_wicket_taken_1over         0.023362
      Opponent_Kenya                 0.021576
      player_highest_wicket          0.014722
      Min_run_given_1over            0.012398
      Opponent_Srilanka              0.011646
      Match_light_type_Day and Night 0.009251
      Opponent_Pakistan              0.008228
      Avg_team_Age                   0.008143
      Opponent_Australia             0.007570
      Season_Winter                  0.007542
      Match_format_T20               0.007477
      Match_format_ODI               0.006206
      Match_light_type_Night         0.004860
      Opponent_England               0.004638
      First_selection_Bowling        0.004073
      Opponent_Zimbabwe              0.002662
      First_selection_Batting        0.002413
      Opponent_Bangladesh            0.001600
      Season_Rainy                   0.001450
```

As can be seen before pruning, **Audience_number** is having the highest importance, followed by **player_highest_run**.

- Train Data and Test Data overall accuracy before pruning:

We will now find the overall accuracy of the train and test data using the model that we just built.

```
[19]: #Train Data
      dt_model.score(X_train,Y_train)
```

```
[19]: 1.0
```

```
[20]: #Test Data
      dt_model.score(X_test,Y_test)
```

```
[20]: 0.9101251422070534
```

The accuracy on the Training Data is 100% and the accuracy on the Test Data is lesser - 91%. The model has surely been overfitted. Thus, we need to prune or regularize the tree.

- As we can see from above images, the training accuracy is 100% and the test accuracy is 91% which signifies that there is obvious overfitting in the model. Thus, we will be reducing by performing pruning of the model

**Iteration 1: Decision Tree with Pruning applied:**

- We set the first set of conditions for the tree to be max_depth=8, min_samples_leaf=20, min_samples_split=10.

## 2. Method 1 for Pruning:

```
[21]: reg_dt_model = tree.DecisionTreeClassifier(criterion = 'gini',
                                                 max_depth=8,#upto this depth is where the tree has grown uniformly
                                                min_samples_leaf=20, #ensures that every terminal node (leaf node) have at least 10
                                                #observations in it
                                                min_samples_split=10) #for every node to be split into two child nodes that particular
                                                #node should have at least 30 observations

[22]: reg_dt_model.fit(X_train, Y_train)

[22]: DecisionTreeClassifier
      ▸ Parameters
      ▸ Fitted attributes

[23]: pd.Series(reg_dt_model.feature_importances_,index=X_train.columns).sort_values(ascending=False)

[23]: Audience_number                0.250136
      Offshore_No                    0.158075
      Opponent_South Africa          0.099845
      Season_Summer                  0.097048
      Extra_bowls_bowled             0.082491
      player_highest_run             0.066610
      All_rounder_in_team            0.054563
```

- Feature Importance

```
[23]: pd.Series(reg_dt_model.feature_importances_,index=X_train.columns).sort_values(ascending=False)

[23]: Audience_number                0.250136
      Offshore_No                    0.158075
      Opponent_South Africa          0.099845
      Season_Summer                  0.097048
      Extra_bowls_bowled             0.082491
      player_highest_run             0.066610
      All_rounder_in_team            0.054563
      extra_bowls_opponent           0.052512
      Max_run_scored_1over           0.028674
      Opponent_Kenya                 0.027832
      Match_light_type_Day and Night 0.019416
      Max_wicket_taken_1over         0.016665
      Min_run_scored_1over           0.015779
      Season_Rainy                   0.014257
      Opponent_Srilanka              0.009988
      First_selection_Bowling        0.005738
      Players_scored_zero            0.000371
      Avg_team_Age                   0.000000
      Wicket_keeper_in_team          0.000000
      Bowlers_in_team                0.000000
      Max_run_given_1over            0.000000
      Min_run_given_1over            0.000000
      Match_format_Test              0.000000
      Match_format_T20               0.000000
      Match_format_ODI               0.000000
      Match_format_20-20             0.000000
      Match_light_type_Night         0.000000
      Match_light_type_Day           0.000000
      player_highest_wicket          0.000000
      First_selection_Batting        0.000000
```

**Audience number** has still highest importance but the second most important variable is now **Offshore_No.**

- Train Data confusion matrix and model metrics

```
[28]: tn, fp, fn, tp = metrics.confusion_matrix(Y_train,Y_train_predict_class).ravel()
      print('True Negative:',tn,'\n''False Positives:' ,fp,'\n''False Negatives:', fn,'\n''True Positives:', tp)

      True Negative: 1687
      False Positives: 35
      False Negatives: 214
      True Positives: 115
```

Let us now go ahead and print the classification report to check the various other parameters.

```
[29]: print(metrics.classification_report(Y_train,Y_train_predict_class))

                   precision    recall  f1-score   support

                0       0.89      0.98      0.93      1722
                1       0.77      0.35      0.48       329

         accuracy                           0.88      2051
        macro avg       0.83      0.66      0.71      2051
     weighted avg       0.87      0.88      0.86      2051
```

- Test Data confusion matrix and model metrics

```
[30]: print(metrics.confusion_matrix(Y_test,Y_test_predict_class),'\n')
      tn, fp, fn, tp = metrics.confusion_matrix(Y_test,Y_test_predict_class).ravel()
      print('True Negative:',tn,'\n''False Positives:' ,fp,'\n''False Negatives:', fn,'\n''True Positives:', tp)

      [[703  32]
       [108  36]]

      True Negative: 703
      False Positives: 32
      False Negatives: 108
      True Positives: 36
```

Let us now go ahead and print the classification report for the test data and compare between train and test.

```
[31]: print(metrics.classification_report(Y_test,Y_test_predict_class))

                   precision    recall  f1-score   support

                0       0.87      0.96      0.91       735
                1       0.53      0.25      0.34       144

         accuracy                           0.84       879
        macro avg       0.70      0.60      0.62       879
     weighted avg       0.81      0.84      0.82       879
```

- Now the training accuracy is 88% and the test accuracy is 84% which signifies that overfitting in the model was reduced. We will try to reduce overfitting further by changing the pruning parameters.

**Iteration 2: Decision Tree with Pruning applied**

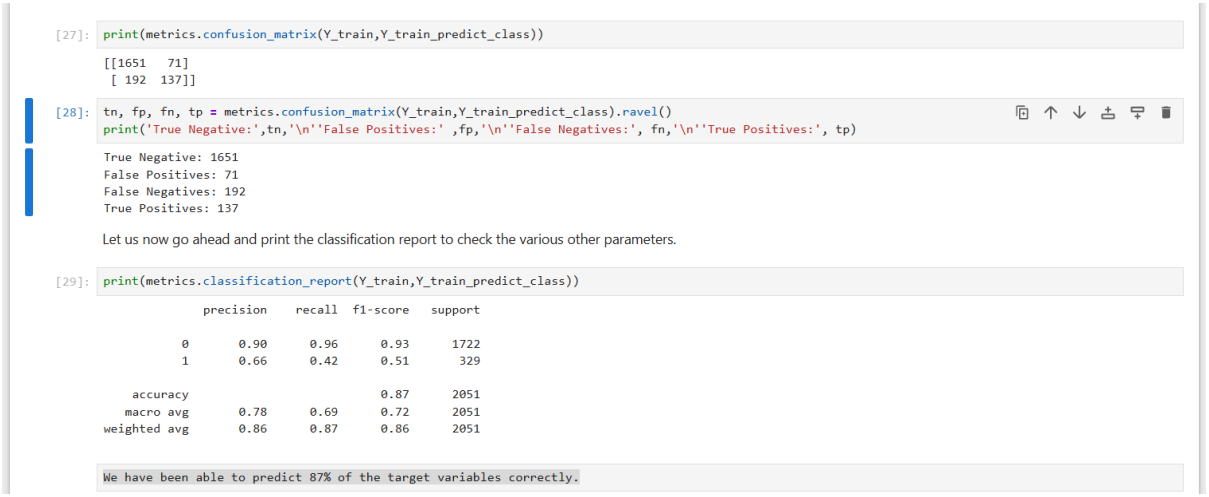
- We will now apply pruning by changing the parameters to max_depth=13, min_samples_leaf=10, min_samples_split=30.
- Feature importance

```
[23]: pd.Series(reg_dt_model.feature_importances_,index=X_train.columns).sort_values(ascending=False)

[23]: Audience_number                  0.258917
      Offshore_Yes                     0.171362
      Season_Summer                    0.105205
      Opponent_South Africa            0.099957
      Extra_bowls_bowled               0.073210
      extra_bowls_opponent             0.071057
      player_highest_run               0.063008
      All_rounder_in_team              0.058730
      Min_run_scored_1over             0.017105
      player_highest_wicket            0.017101
      Match_light_type_Day and Night   0.012693
      Max_wicket_taken_1over           0.010843
      Season_Rainy                     0.009736
      Match_format_ODI                 0.008748
      First_selection_Bowling          0.006776
      Avg_team_Age                     0.004698
      First_selection_Batting          0.003990
      Min_run_given_1over              0.003961
      Opponent_Kenya                   0.002670
      Max_run_scored_1over             0.000233
      Wicket_keeper_in_team            0.000000
      Bowlers_in_team                  0.000000
      Match_format_Test                0.000000
      Match_format_T20                 0.000000
      Match_format_20-20               0.000000
      Match_light_type_Night           0.000000
      Match_light_type_Day             0.000000
      Players_scored_zero              0.000000
      Max_run_given_1over              0.000000
      Opponent Bangladesh              0.000000
```

**Audience number** has still highest importance but the second most important variable is now **Offshore_Yes**.

- Train Data confusion matrix and model metrics



```
[27]: print(metrics.confusion_matrix(Y_train,Y_train_predict_class))

[[1651   71]
 [ 192  137]]

[28]: tn, fp, fn, tp = metrics.confusion_matrix(Y_train,Y_train_predict_class).ravel()
      print('True Negative:',tn,'\n''False Positives:' ,fp,'\n''False Negatives:', fn,'\n''True Positives:', tp)

True Negative: 1651
False Positives: 71
False Negatives: 192
True Positives: 137
```

Let us now go ahead and print the classification report to check the various other parameters.

```
[29]: print(metrics.classification_report(Y_train,Y_train_predict_class))

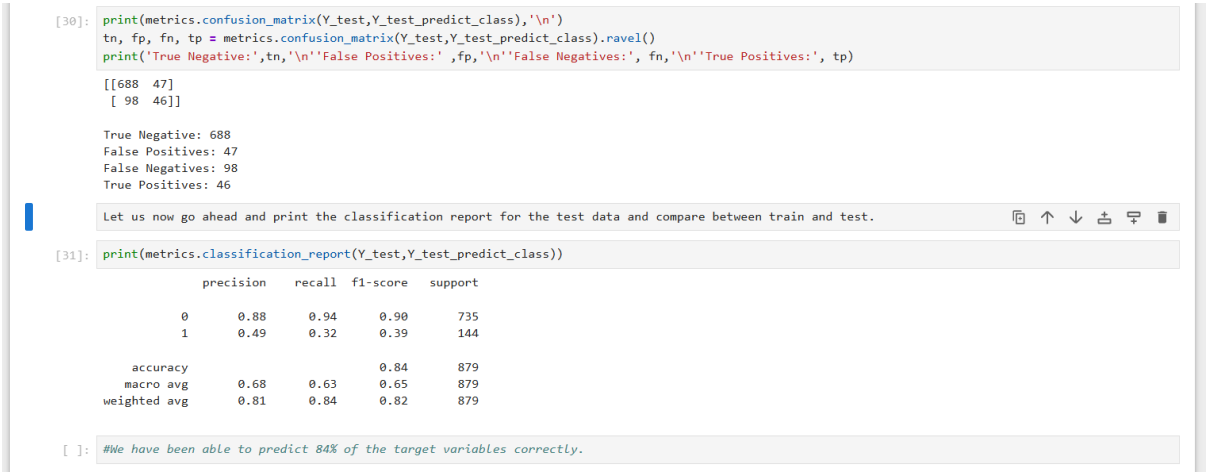
              precision    recall  f1-score   support

           0       0.90      0.96      0.93      1722
           1       0.66      0.42      0.51       329

    accuracy                           0.87      2051
   macro avg       0.78      0.69      0.72      2051
weighted avg       0.86      0.87      0.86      2051
```

We have been able to predict 87% of the target variables correctly.

- Test Data confusion matrix and model metrics



```
[30]: print(metrics.confusion_matrix(Y_test,Y_test_predict_class),'\n')
      tn, fp, fn, tp = metrics.confusion_matrix(Y_test,Y_test_predict_class).ravel()
      print('True Negative:',tn,'\n''False Positives:' ,fp,'\n''False Negatives:', fn,'\n''True Positives:', tp)

[[688  47]
 [ 98  46]]

True Negative: 688
False Positives: 47
False Negatives: 98
True Positives: 46
```

Let us now go ahead and print the classification report for the test data and compare between train and test.

```
[31]: print(metrics.classification_report(Y_test,Y_test_predict_class))

              precision    recall  f1-score   support

           0       0.88      0.94      0.90       735
           1       0.49      0.32      0.39       144

    accuracy                           0.84       879
   macro avg       0.68      0.63      0.65       879
weighted avg       0.81      0.84      0.82       879

[ ]: #We have been able to predict 84% of the target variables correctly.
```

- As we can see, the training accuracy is 87% and the test accuracy is 84% accurate which signifies that overfitting in the model was further reduced.

**Conclusion and Inferences.**

- The model has reduced both type 1 error and type 2 error in the test set as seen in the above image.
- The training and test set accuracy is very close which means that there is almost no overfitting.
- Thus, we can ensure that the model can predict whether India will win or lose the match about 80% to 85 % of the time.
- This means that the Indian Cricket Team's management can save time and resources and take the correct decisions which leads to a win by the Cricket Team.

**Recommendations:**

- The Indian Cricket Team's management should use the model but also should have plan B as there is about 10%-15% chance that the model can provide the wrong predictions.

- The Indian Cricket Team should also try and test other models like Random Forest which is known to produce more accurate results than a single decision tree. This will help them to take decisions more effectively.